01 · App Overview — BetConstructAI

> Export for the mobile team rebuilding this product natively (Flutter, iOS + Android). Source of truth: the demo code in this repo. Nothing here is invented; unknowns live in `OPEN_QUESTIONS.md`.

What the app is (one paragraph)

BetConstructAI is a **B2B iGaming marketplace and sales/CRM platform** built around the BetConstruct ecosystem (sportsbook, casino, crypto-gaming, platforms, payments, licenses, affiliate/marketing). It has two faces in one product:

1. A **public client app** (`index.html`) where iGaming operators, game/payment providers, affiliates and buyers browse a catalog of BetConstruct solutions and third-party vendor products, get a **configuration recommendation from an AI advisor**, submit partner/vendor applications, add items to an "interest cart", and **chat live with a manager**.
2. An internal **team back-office** (`admin.html` for admins, `staff.html` for sales/staff) — an all-in-one CRM that replaces ~5 separate tools (live chat, internal comms, sales pipeline, tasks, e-signature): applications moderation, marketplace vendor management, CRM deals pipeline, agreements, tasks, omnichannel communications, visitor analytics, and content.

The problem it solves: give BetConstruct a **single platform** to attract, qualify, advise and convert B2B partners — from first anonymous visit, through AI-assisted solution selection and manager chat, to a tracked deal and signed agreement — instead of stitching together Hoory (live chat), Pandayo (internal comms), Zoho (CRM), Jira (tasks) and DocuSign (agreements).

> **Important for store submission:** this is a **B2B business/SaaS tool** — a catalog, an advisor and a CRM. It does **not** take real-money bets, run casino play or process player deposits. That distinction is what keeps it out of the strict "real-money gambling app" store rules.

Who uses it (roles)

Role	Where they are	What they want
Visitor / Guest (end user)	Client app (`index.html`)	Understand BetConstruct solutions, get an AI recommendation for their case, request a demo, reach a manager. Enters with just a name + contact ("Войти как гость") — no password.
Prospect / Applicant (partner, vendor, affiliate, buyer)	Client app → registration form	Register their company/product to be listed in the marketplace or to become a client.
Sales / Staff	Staff cabinet (`staff.html`)	See who is online, own leads, own clients/deals, agreements, tasks, chats, their shifts and analytics. Log in with login+password.
Admin	Admin panel (`admin.html`)	Everything staff sees plus: moderate applications, manage marketplace vendors, run the full CRM, manage the team, agreements, tasks, documents, analytics, content. Log in with email+password.
Partner	*(REMOVED)* Formerly `partner.html`	A partner cabinet existed (partners logging in to see their leads/deals/agreements) but was **removed** on the owner's decision. Partner login is gone; `partner.html` deleted. See OPEN_QUESTIONS.

There is now a **single unified login** at `/login` (`login.html`) that authenticates **admins and staff** and routes each to the right cabinet. Regular users continue via the guest entry in the client app.

Main "jobs" a user does (in order of importance)

Client app (visitor):

1. **Enter as guest** — give name + email/phone to start (no account).
2. **Browse the catalog** — BetConstruct solutions (Turnkey / Crypto / White Label / API iGaming), platforms, products, regions, licenses, payments; and marketplace **vendor** products.
3. **Ask the AI advisor** — describe a task (geo, vertical, crypto/fiat, budget) → get a ready configuration (solution + region/license + vendor products + payments), grounded in the real catalog.
4. **Add to interest cart** and **request a demo** for specific products.
5. **Chat with a manager** (live, polling) — including receiving a background nudge when a manager replies while chat is closed.
6. **Register** a partner/vendor/affiliate/buyer application.

Back-office (staff / admin):

1. **See who is online** and message visitors in real time.
2. **Work leads** → move **deals** through the pipeline (6 stages + a 7-step operational flow).
3. **Moderate applications** and publish **marketplace vendors** with badges.
4. **Manage agreements** (draft → sign → pay), **tasks**, **documents**, **omnichannel comms**.
5. **Read analytics** (marketplace funnel, visitor paths, sales results, shift ranking).
6. (Admin) **Manage the team**, **content**, **accounts registry**.

Core concepts / vocabulary (define every domain noun the UI uses)

- **Solution (SOL)** — a top-level BetConstruct offering: *Turnkey iGaming, Crypto iGaming, White Label iGaming, API iGaming*.
- **Platform (PLAT)** — e.g. *CMS Pro, SpringBuilderX*.
- **Product / PCAT item** — individual products/services grouped by category (Sportsbook, Casino, Live Casino, etc.).
- **Pack (PACKS)** — priced bundles (e.g. White Label Curaçao — €19 900).
- **BME / Backoffice** — BetConstruct's management console modules.
- **Region / License** — jurisdictions and licenses (Curaçao, Malta, France, UK…).
- **Payments (PAY)** — fiat + crypto payment options / gateways.
- **Vendor** — an approved third-party product listed in the marketplace (from an approved application). Has a **badge**: `pending → approved → partner → new`, and metrics (impressions, clicks, match_score).
- **Application** — a partner/vendor/affiliate/buyer registration submitted from the client app; the admin **moderates** it (status: pending / approved / partner / new / rejected) and can **activate a portal** for it (historically → partner cabinet).
- **Interest cart** — the visitor's saved list of products of interest (client-side).
- **Lead** — an inbound interest signal: source `demo_request | cart | chat`, with visitor_name + contact.
- **Deal** — a sales opportunity in the CRM (id like `BC-1042`), with responsible people by role (resp / ssales / pm / am), a **stage** (6 Zoho-like stages) and a **7-step flow** (`flow` jsonb array), plus `steps`, `rr_done`, `kick_done`.
- **Agreement** — a document tied to a deal: draft → signed → paid (replaces DocuSign).
- **Task** — a task/resource request (replaces Jira): title, dept, assignee, priority, status.
- **Account** — an entry in the CRM registry (client/partner/prospect) with an owner.
- **Contact / comm_message** — omnichannel conversation records tied to accounts.
- **Thread / thread_message** — chat channels (`kind`: visitor / kickoff / internal) and their messages (`role`: visitor / staff / partner).
- **Event** — a visitor tracking event (`type`: page, click, cart, lead, ai, search, entry, ping).
- **Visitor** — a device profile (`anon_id`, device json) — only stored with consent.
- **Shift** — a staff work shift (for the presence/ranking features).

- **AI advisor** — the conversational recommender ("Ани, менеджер BetConstruct" persona in mock chat; the advisor endpoint is a separate LLM proxy).
- **Guest** — an unauthenticated visitor identified by a first-party `anonId` (used as chat `guest_key`).
- **VPN bar** — a **simulated** VPN toggle strip at the top of the client app (cosmetic demo feature; not a real VPN).

What is REAL vs MOCKED in this demo

Area	State
Data storage	**Real** — Supabase (Postgres + REST + Storage). Project ref `smddtvaewmmpuyvtuscb`. The browser reads/writes via a public **anon (publishable) key**.
Auth	**Partly real / weak.** No real identity provider (no Supabase Auth/JWT). "Session" = a `sessionStorage` flag. Passwords were plaintext in the DB; login is now verified **server-side** by `POST /api/login` (service_role) and passwords are **column-REVOKE'd** from the anon key. Guest "login" is just a name+contact stored locally.
AI advisor	**Real endpoint, provider in flux.** `POST /api/advisor` proxies to an LLM. **The code in THIS repo targets Google Gemini** (`gemini-2.5-flash` , `GEMINI_API_KEY`) and currently fails (project denied / model deprecated), so the client falls back to a local canned answer (`aiLocal`). A rewrite to **Anthropic Claude Haiku 4.5** was authored out-of-repo and is **not present here or deployed** (see `05_DATA_AND_API.md` and OPEN_QUESTIONS). Treat the advisor as: same `{prompt, context, lang} → {text}` contract, provider undecided. The in-app "manager chat" opener uses a **mock** greeting ("Ани, менеджер BetConstruct").
Payments	**None.** No real payment processing anywhere. Prices are static catalog text. Agreement "pay" is a status only.
Notifications	**In-app only.** No push. A background poll shows a toast when a manager replies. `/api/notify` posts internal notifications; email via Gmail is coded but not connected.
VPN bar	**Faked** — simulated servers/IPs, cosmetic.
FingerprintJS + analytics	**Real** but **consent-gated** (GDPR banner). Off until the user taps "Принять".
Marketplace / catalog	**Mixed** — BetConstruct solutions/platforms/products are **static in code**; vendors/applications are **real DB rows** (currently mostly demo/test rows like "Test", "Dmer").
Gmail two-way email	**Coded, not connected** (corporate Google blocks the OAuth app).

Tech stack of this demo (so you know what you're reading)

- **Frontend:** single-file HTML "SPA" per screen — `index.html` (client, ~~1.4k lines),~~ ~~`admin.html`~~ (1.1k lines), `staff.html`, `login.html`, `privacy.html`. No build step, no framework. Each file has inline `<style>` and `<script>`; routing is a JS `go(page)/render()` function; i18n is an in-file dictionary + `translateDOM()`. Vanilla JS + `supabase-js` (UMD from CDN) + FingerprintJS (CDN).
- **Backend: Vercel serverless functions** in `api/*.js` (ESM `export default handler`, plain `fetch`). Key ones: `api/login.js` (server-side auth), `api/advisor.js` (LLM proxy), `api/notify.js`, `api/comm-ingest.js`, `api/email-inbound.js`, `api/gmail-*.js`.
- **Database/Storage/Auth: Supabase** (Postgres 17, REST, Storage bucket `uploads`). RLS was historically open (`using(true)`); password columns are now column-level REVOKE'd from `anon` / `authenticated`.
- **Realtime:** none — **polling** everywhere (chat ~3.5s, presence ~20s, background chat watch ~6s, heartbeat ping ~45s).
- **Hosting:** Vercel. Live at `betconstruct-ai.vercel.app` / `betconstructai.com`. Deploy repo: GitHub `mantarlyani/betconstructAI`.
- **Device id:** FingerprintJS v4 (CDN) → stable `anonId` in `localStorage` (`bc_anon`), used as chat `guest_key` and analytics key.
- **Languages:** RU (primary), EN, HY (Armenian). All LTR.

> The mobile team will **not** reuse this code — treat this as the functional/visual spec, not an implementation reference.